hana Club Website | Native Event Registration & Form Builder

lementary Document v1.0

1. Change Request Overview

This document serves as an official change request and supplementary specification to the original Anveshana Club project documents. It details the replacement of the existing external event-registration mechanism (Google Forms / WhatsApp) with a fully native, dynamic registration system integrated directly into the website.

The scope includes a dynamic form builder for administrators, event-specific participant management, CSV/XLSX exports, and automated form versioning to preserve historical registration data.

2. Scope and Preservation Rules

> **CRITICAL INSTRUCTION FOR AI AGENTS & DEVELOPERS:** This document **DOES NOT** replace the original six project documents. Treat the original PRD, TRD, Application Flow, UI/UX, Backend Schema, and Implementation Plan as the immutable baseline. Introduce only the functionality specified below.

- Preserve the existing technology stack, visual identity, and routing architecture.
- Do NOT introduce student authentication or login walls. Public access remains exactly as originally specified.
- Do NOT introduce payment gateways, QR code attendance, or SMTP email ticketing.
- If this CR conflicts with original specifications regarding Google Forms, this document takes precedence.

3. Change Impact Matrix

Original Document	Impact / Additions Required
1. PRD	Added sections for Native Registration, Form Builder, and Participant Management.
2. TRD	Added requirements for JSON Schema form generation, versioning logic, and server validation.
3. Application Flow	New public registration flow. New admin form creation and participant export flow.
4. UI/UX Design Brief	Addition of Admin Form Builder interface and Public Dynamic Form renderer components.
5. Backend Schema	Added `Registrations` collection. Added `formSchema` and `formVersion` to `Events` collection.
6. Implementation Plan	Appended 11 incremental tasks specific to this change request.

4. New Functional Requirements

Req ID	Feature	Requirement Description
CR-FR-001	Native Event Registration	Students must be able to register natively on the website by completing a dynamic form loaded on the Event Details page. No external redirects permitted.
CR-FR-002	Admin Form Builder	Administrators can independently configure fields per event (Add, Edit, Remove, Reorder, Enable/Disable).
CR-FR-003	Supported Field Types	Builder must support: Short text, Long text, Email, Phone number, Number, Dropdown/Select, Radio buttons, Checkboxes, Date.
CR-FR-004	Form Versioning	Form edits must not corrupt existing registrations. The system must preserve historical answers against a specific schema version.
CR-FR-005	Participant Dashboard	Admins must have an event-specific view showing total registrations, a participant table, and individual details. Includes search, filter, and pagination.
CR-FR-006	Data Export	Admins can export an event's participants to CSV (mandatory) and XLSX (if stack-compatible), preserving custom field labels.
CR-FR-007	Submission Confirmation	Upon successful backend persistence, the UI displays a confirmation message with a unique registration reference ID.

5. New Non-Functional & Security Requirements

Req ID	Category	Requirement Specification
CR-NFR-001	Data Integrity	The backend must independently validate all form submissions against the active, stored server-side form definition, rejecting unknown fields or altered choice payloads.
CR-NFR-002	Concurrency & Limits	The backend must handle concurrent submissions near participant limits accurately, ensuring the configured limit is never exceeded by simultaneous requests.
CR-NFR-003	Export Security	CSV/XLSX generation must sanitize input to mitigate spreadsheet formula injection (e.g., escaping cells starting with `=`, `+`, `-`, `@`).
CR-NFR-004	Privacy	Public APIs must never expose participant PII or full participant lists. Admin participant routes must require strict session validation.

6. Dynamic Form Builder Specification

The form builder is integrated directly into the Admin Event Management interface. It generates a JSON schema defining the registration requirements for a specific event.

- **Field Properties:** Every field must possess a stable unique identifier (e.g., UUID or hashed ID), field type, text label, display order, enabled status, and a required/optional boolean flag.
- **Optional Properties:** Help text, placeholders, min/max values (for numbers/dates), text-length limits, and a predefined array of `choices` (for dropdowns, radios, and checkboxes).
- **Standard Fields:** Base fields (Name, Student ID, Email, Phone, Department) are **not hardcoded** at the database level. They are treated as default fields added to a new event's schema but can be modified, disabled, or marked optional by the admin based on the event's needs.
- **State Management:** Admins can preview the form before publishing. Only publishing a form makes it active on the public page.

> **Destructive Action Warning:** If an administrator attempts to remove or disable a field that already has associated responses, the UI must prompt a severe warning. The backend must enforce that "removing" a field simply disables it in the active schema, preserving historical data ties.

7. Registration Lifecycle and Validation Rules

The registration lifecycle introduces new states and hard constraints to ensure data quality and operational control.

- **Validity Checks:** Submissions are immediately rejected if the associated event is unpublished, archived, or deleted.
- **Time Windows:** Submissions are rejected if the current server time falls outside the configured registration open/close timestamps.
- **Capacity Limits:** If a `participantLimit` is set, the server must run a transaction/lock to verify capacity before inserting the record. Once reached, the event automatically transitions to a "Full" state.
- **Duplicate Prevention:** A default deduplication rule is applied per event. Default rule: *1 registration per event per Student ID* (if Student ID is enabled). If Student ID is disabled, fallback to Email. Duplicate attempts must return a clear, polite error preventing submission.
- **Transactional Persistence:** Success messages are strictly gated behind successful database commits. Failed commits must yield explicit error messages.

8. Form Versioning & Historical Data Preservation

Because forms are dynamic, changing a field (e.g., altering a Dropdown to a Text Input, or removing a question) risks corrupting data submitted prior to the change. To prevent this, the architecture will utilize **Form Versioning**.

- **Snapshotting:** When an event is created, its form schema is initialized at `version: 1`.
- **Submitting:** When a student registers, their payload includes the answers AND the specific `formVersion` they filled out. The database binds this version to the registration record.
- **Updating:** If an admin edits and publishes the form, the `formVersion` integer increments on the Event document. Existing registrations remain tied to older versions.
- **Field Immutability:** To prevent data orphans, renaming a label only changes the visual text; the underlying stable field identifier remains identical.
- **Rendering Admin Data:** When viewing or exporting historical registrations, the backend resolves the answers using the schema version tied to that specific registration, ensuring headers and data types align correctly.

9. Participant Management and Export Specification

The admin dashboard will feature a new **Participant Management Page** scoped strictly to the selected event context.

Data Display & Interactions

The view renders a data table mapping the active form schema's labels as column headers. It includes global search across text fields, and sorting by registration timestamp. Clicking a row opens an individual detail view to inspect all custom answers.

Export Operations

- **CSV Export (Mandatory):** Triggering an export generates a `.csv` file scoped only to the selected event. Headers dynamically match the event's form configuration. Commas, quotes, and line breaks inside custom text fields must be strictly escaped. The file must be named `[Event_Slug]_Participants_[Date].csv`.
- **XLSX Export (Conditional):** If compatible with the existing backend framework (e.g., utilizing `xlsx` or `exceljs` in Node.js), a native Excel export must be provided alongside CSV, ensuring columns auto-fit and formulas are sanitized via prepended apostrophes (`'`).

10. Database Schema Additions and Modifications

The following extensions must be integrated into the existing backend schema (referencing Document 5: Backend Schema Document). Do not create duplicate Event tables.

Modified Collection: `Events`

Field	Type	Description / Update
`formSchema`	JSON Array	**NEW.** Ordered array of field configuration objects (id, type, label, required, choices, etc.).
`formVersion`	Integer	**NEW.** Increments on every form publication. Defaults to 1.
`regOpenDate`	Timestamp	**NEW.** Start time for accepting registrations.
`regCloseDate`	Timestamp	**NEW.** End time for accepting registrations.
`maxParticipants`	Integer	**NEW.** Optional limit. NULL implies unlimited.
`deduplicationRule`	String	**NEW.** Defines the unique constraint field (e.g., "studentId" or "email").

New Collection: `Registrations`

Field	Type	Description
`id`	UUID/String	Primary Key. Used as the public Reference Number shown to students.
`eventId`	FK	Reference to the associated Event document. Indexed for fast lookup.
`formVersion`	Integer	The snapshot version of the schema this registration was submitted against.
`answers`	JSON Object	Key-value pairs mapping stable field identifiers to the participant's input.
`identifier`	String	Extracted deduplication value (e.g., the actual Student ID) used for quick uniqueness checks based on the `deduplicationRule`.
`submittedAt`	Timestamp	Server-generated timestamp of successful commit.

11. New and Modified API Contracts

Ensure these endpoints utilize the same authentication middleware and error-response formatting as defined in the original TRD and Application Flow.

Method & Route	Auth	Contract Details
`GET /api/v1/events/:id/form`	Public	**Req:** Event ID. **Res:** Returns `formSchema` (if event is published and open), `formVersion`, and status flags (`isOpen`, `isFull`).
`POST /api/v1/events/:id/register`	Public	**Req:** `formVersion` integer, `answers` JSON payload. **Validation:** Server validates `answers` against `formSchema` at specified version. Checks capacities and dupes. **Res:** `201 Created` with Reference ID, or `400/409` for validation/dupe errors.
`PUT /api/v1/admin/events/:id/form`	Admin Only	**Req:** Complete new `formSchema` array. **Behavior:** Increments `formVersion`, updates schema on Event document. **Res:** `200 OK` with updated version ID.
`GET /api/v1/admin/events/:id/registrations`	Admin Only	**Req:** Query params `page`, `limit`, `search`. **Res:** Paginated list of Registrations containing parsed `answers`, total count, and applied form version.
`GET /api/v1/admin/events/:id/export`	Admin Only	**Req:** Query param `format` (csv/xlsx). **Res:** File stream with appropriate `Content-Type` and `Content-Disposition` attachment headers.

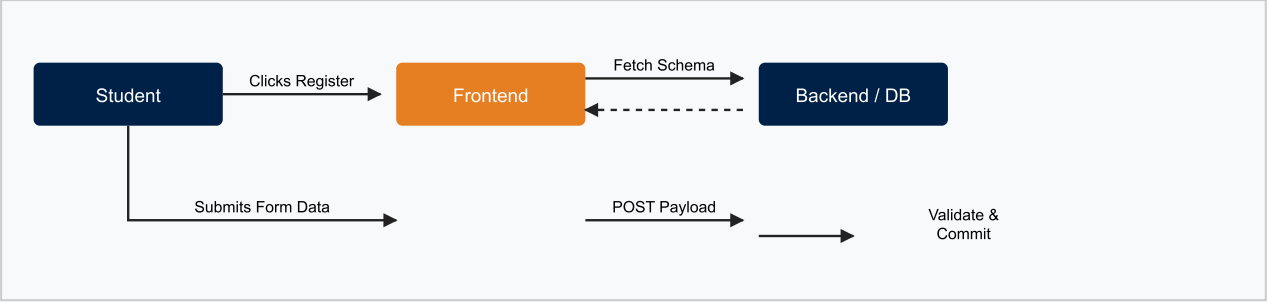
12. UI/UX Additions

UI components must strictly follow the existing UI/UX Design Brief (navy, gold, orange, serif headers, modern inputs). Do not redesign unrelated pages.

- **Public - Form Renderer:** A dynamic React/Vue component that maps JSON field definitions to native HTML inputs. Features inline validation styling (red borders on error) and required asterisks.
- **Admin - Builder:** A drag-and-drop or up/down arrow interface for reordering fields. A property panel appears when a field is selected to edit labels, choices, and toggles.

13. Updated Application Flows (Mermaid)

Native Registration Sequence



Developer Reference (Mermaid):

```
sequenceDiagram
    participant Student
    participant WebUI as Frontend
    participant API as Backend (Node/DB)
    Student->>WebUI: Clicks "Register" on Event Page
    WebUI->>API: GET /api/v1/events/:id/form
    API-->>WebUI: Returns JSON Schema & active status
    WebUI-->>Student: Renders Dynamic Form
    Student->>WebUI: Fills form & Submits
    WebUI->>API: POST /api/v1/events/:id/register (answers, version)
    API->>API: Validate against Schema Snapshot
    API->>API: Check Capacity & Dupes
    API->>DB: Transaction Commit
    API-->>WebUI: 201 Created (Ref ID)
    WebUI-->>Student: Success Screen Displayed
```

14. Incremental Implementation Plan

Execute this change request in the following dependency order, appending to the original Implementation Plan:

1. **Inspect:** Review original 6 documents to ensure tech-stack alignment (React/Node/DB).
2. **Schema Update:** Apply DB migrations for `Registrations` and new `Events` fields.
3. **Backend Configuration:** Implement `formSchema` validation and `formVersion` logic.
4. **Admin API:** Build endpoints for creating/updating form schemas.
5. **Builder UI:** Build the drag-and-drop form configuration interface in the Admin Dashboard.
6. **Public API:** Build endpoints to fetch schema and submit registrations.
7. **Renderer UI:** Implement public dynamic form rendering.
8. **Persistence:** Wire submission handling, duplication checks, and capacity locks.
9. **Management UI:** Build Admin Participant table, individual details view, and pagination.
10. **Export:** Implement CSV generation (and XLSX if compatible).
11. **Testing:** Run full regression and security validations against the CR test cases.

15. Acceptance Criteria and Test Cases

Test ID	Scenario	Expected Result
CR-TEST-001	Dynamic Field Rejection	Submit a POST payload with a field identifier not present in the schema. System returns `400 Bad Request`.
CR-TEST-002	Schema Versioning	Admin removes a field. Submit new registration. Verify historical registrations still show the old field, new ones do not.
CR-TEST-003	Capacity Concurrency	Send 10 simultaneous requests to an event with capacity=1 remaining. 1 succeeds, 9 fail with event full error.
CR-TEST-004	Deduplication	Submit with an existing Student ID. Backend rejects with duplicate error.
CR-TEST-005	Export Security	Register user with name `=-1+2+cmd|' /C calc'!A0`. Export to CSV. Verify prefix is escaped to prevent execution.

16. Dependencies and Compatibility

- **Authentication:** Relies on the Admin Auth mechanism defined in PRD Section 8 / TRD Section 3. No changes to Auth.
- **UI Components:** Relies on buttons, cards, and modal definitions from UI/UX Design Brief. Inputs must inherit standard border/focus styling.
- **API Architecture:** Must conform to existing RESTful conventions defined in TRD.

17. Unresolved Decisions Requiring Approval

- **Data Retention Policy:** Does deleting an Event permanently hard-delete its associated Registrations, or softly archive them? (Pending Product Owner approval).
- **XLSX Practicality:** The AI Agent / Dev must confirm if adding an Excel parsing library exceeds deployment size budgets. If yes, fallback to CSV-only is pre-approved.

18. Cross-Feature Consistency Checklist

- Event Details page correctly hides the "Register via Google Forms" button and mounts the new React/Vue form renderer.
- Admin Event creation flow correctly initializes a default `version: 1` schema.
- Public endpoints remain accessible without a JWT token.
- Export routes are strictly protected by Admin JWT middleware.

19. Change Log

Version 1.0 (Current) - Initial Change Request drafted for Dynamic Event Registration Form Builder and Participant Management. Supersedes all references to external form providers in baseline documentation.

End of Supplementary Specification. Proceed to implementation utilizing this document in conjunction with baseline specs.